

Práctica No. 3: Gestión de Memoria

Integrantes

Nombre: Cristian Flórez

Correo: cristian.florez@udea.edu.co

Documento: 1040183014

Video de sustentación: <https://youtu.be/aN0a4jwfOKs>

1. Espacio de direcciones

1.2 Actividad: Visualizar los mapas de memoria de un proceso

Mientras el programa espera el ENTER, abra una segunda terminal:

```
cristian@DESKTOP-5T8LL5D:/mnt/c/users/crist/OneDrive/Escritorio/Udea/SO/Labs/Lab03-SO-Cristian-Florez$ ./mem_map.exe
PID del proceso : 2424
Dir. codigo (main) : 00007FF6D80D1450
Dir. global_var : 00007FF6D80D3000
Dir. local_var : 0000000F58FFFD94
Dir. heap_var : 000001797EE29A70
Presione ENTER para continuar...
```

Leer el mapa de memoria del proceso

cat /proc/\$(pgrep mem_map)/maps

```
cristian@DESKTOP-5T8LL5D:/$ cat /proc/$(pgrep mem_map)/maps
00200000-002c3000 r--p 00000000 00:00 28003 /init
002c3000-002c4000 r--p 000c3000 00:00 28003 /init
002c4000-00495000 r-xp 000c3000 00:00 28003 /init
00495000-00496000 r-xp 00294000 00:00 28003 /init
00496000-004a9000 rw-p 00294000 00:00 28003 /init
004a9000-004aa000 rw-p 002a6000 00:00 28003 /init
004aa000-004b3000 rw-p 00000000 00:00 0
007bc000-007bd000 ---p 00000000 00:00 0 [heap]
007bd000-007be000 rw-p 00000000 00:00 0
7f50b5500000-7f50b5502000 rw-p 00000000 00:00 0
7f50b5510000-7f50b5512000 rw-p 00000000 00:00 0
7f50b5520000-7f50b5521000 rw-p 00000000 00:00 0
7f50b5530000-7f50b5531000 rw-p 00000000 00:00 0
7f50b5540000-7f50b5541000 rw-p 00000000 00:00 0
7f50b5550000-7f50b5551000 rw-p 00000000 00:00 0
7f50b5560000-7f50b5561000 rw-p 00000000 00:00 0
7f50b5570000-7f50b5571000 rw-p 00000000 00:00 0
7f50b5580000-7f50b5581000 rw-p 00000000 00:00 0
7f50b5590000-7f50b5591000 rw-p 00000000 00:00 0
7f50b55a0000-7f50b55a1000 rw-p 00000000 00:00 0
7fffe4b47000-7fffe5347000 rw-p 00000000 00:00 0 [stack]
7fffe58d0000-7fffe58d1000 r-xp 00000000 00:00 0 [vdso]
cristian@DESKTOP-5T8LL5D:/$
```

```
# Vista resumen con tamaños de cada región
pmap -x $(pgrep mem_map)
```

1.3 Actividad: Exploración de /proc/[pid]/maps

1. Identifique en la salida de /proc/maps las regiones text, heap y stack. ¿Qué permisos (r/w/x/p) tiene cada una? ¿Por qué difieren?

Región	Rango de Direcciones (Tu salida)	Permisos	Explicación
Código (Text)	00495000-00496000	r-xp	Contiene las instrucciones ejecutables. Tiene permiso x (execute) y r (read), pero no w (write) para evitar que el programa se modifique a sí mismo por seguridad.
Heap	007bc000-007bd000	---p / rw-p	Memoria dinámica asignada por malloc. Tiene permisos r y w porque el proceso necesita leer y escribir datos allí.
Stack	7fffe4b47000-7fffe5347000	rw-p	Almacena variables locales y llamadas a funciones. También es de lectura/escritura (rw).

2. Compare las direcciones impresas con los rangos de /proc/maps. ¿A qué región pertenece cada variable?

Elemento	Dirección Virtual (Hex)	Tipo de Memoria
PID	2424	Identificador del Proceso
main	00007FF6D80D1450	Segmento de Código (Text)
global_var	00007FF6D80D3000	Segmento de Datos (.data)
heap_var	000001797EE29A70	Heap (Memoria Dinámica)
local_var	0000000F58FFFD94	Stack (Pila)

3. ¿Qué otras regiones aparecen en el mapa (libc, [vdso], [vsyscall])? ¿Qué función cumple cada una?

- Bibliotecas Compartidas (libc): Estas regiones se localizan en las direcciones intermedias del mapa (en este caso, con prefijos 7f50...). Se distinguen por poseer permisos de lectura y ejecución (r-xp) y, en entornos Linux nativos, por estar vinculadas a archivos de objetos compartidos (.so).

Función: Proveen la implementación de funciones estándar de C, como printf() y malloc(). El sistema operativo emplea un mecanismo de mapeo para que múltiples procesos compartan una única copia física de estas librerías en RAM, optimizando el uso de los recursos del sistema.

- [vdso] (Virtual Dynamic Shared Object): Se identifica explícitamente a través de la etiqueta de mapeo en las direcciones virtuales superiores (cercanas al stack).

Función: Es una pequeña biblioteca inyectada por el kernel en el espacio de usuario. Su propósito es permitir que ciertas llamadas al sistema (system calls) frecuentes se ejecuten sin la necesidad de un cambio de contexto costoso hacia el modo kernel, mejorando significativamente la eficiencia del proceso.

- [vsyscall]: Se sitúa en el extremo superior del espacio de direcciones virtual.

Función: Constituye un mecanismo heredado para la aceleración de llamadas al sistema. Aunque cumple una función similar al vdso, su uso es limitado en sistemas modernos debido a restricciones de seguridad y flexibilidad, manteniéndose principalmente por razones de compatibilidad.

4. ¿Son las direcciones virtuales iguales a las físicas? Explique apoyándose en el concepto de address space del OSTEP.

Son virtuales ya que el proceso no accede directamente a la RAM. El sistema operativo utiliza una Tabla de Páginas para traducir estas direcciones virtuales a direcciones físicas reales.

1.4 Actividad: Comparar espacios de dos procesos simultáneos

Ejecute dos instancias de mem map al mismo tiempo en dos terminales distintas y compare las direcciones de main, global var y heap var.

Variable	Proceso A (PID 3196)	Proceso B (PID 2424)	Observación
main	00007FF6D80D1450	00007FF6D80D1450	Idénticas

global_var	00007FF6D80D3000	00007FF6D80D3000	Idénticas
heap_var	0000025B3A33D3F0	000001797EE29A70	Distintas

1. ¿Son las mismas direcciones virtuales en ambos procesos? ¿Qué conclusión saca sobre el aislamiento del espacio de direcciones?

Se observa que las direcciones virtuales para el código (main) y las variables globales (global_var) son idénticas en ambos procesos. En el caso del heap, las direcciones difieren debido a la gestión dinámica de memoria y mecanismos de seguridad como ASLR.

2. ¿Podrá el Proceso A leer o modificar la variable global del Proceso B mediante su dirección virtual? Justifique.

No, es imposible que el Proceso A acceda o modifique la memoria del Proceso B utilizando su dirección virtual ya que el acceso a la memoria está estrictamente mediado por la Unidad de Gestión de Memoria (MMU) y el kernel del sistema operativo. Cuando el Proceso A intenta acceder a la dirección 00007FF6D80D3000, el hardware traduce esa dirección basándose únicamente en la tabla de páginas del Proceso A. Dicha tabla solo contiene mapeos hacia marcos físicos autorizados para ese PID específico. Cualquier intento de un proceso por acceder a una dirección virtual que no esté mapeada en su propia tabla de páginas, o que pertenezca al espacio del kernel o de otro proceso, resultará en una excepción de hardware.

2 API de Memoria

Una vez compilado, monitoree la ejecución del programa con Valgrind:

```
gcc -Wall -o heap_demo heap_demo.c
valgrind --leak-check=full --track-origins=yes ./heap_demo
```

2.2 Actividad: Uso correcto de malloc y free

1. Muestre la salida completa de Valgrind. ¿Reporta errores o fugas de memoria? ¿Qué significa el mensaje "All heap blocks were freed"?

```

Cristian@DESKTOP-ST8LL5D:/mnt/c/users/crist/OneDrive/Escritorio/UdeA/50/Labs/Lab03-50-Cristian-Florez$ valgrind --leak-check=full --track-origins=yes ./heap_demo
==373== Memcheck, a memory error detector
==373== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==373== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==373== Command: ./heap_demo
==373==
==373== error calling PR_SET_PTRACER, vgdb might block
Arreglo original: 0 1 4 9 16 25 36 49 64 81
Arreglo ampliado: 0 1 4 9 16 25 36 49 64 81 100 121 144 169 196 225 256 289 324 361
==373==
==373== HEAP SUMMARY:
==373==   in use at exit: 0 bytes in 0 blocks
==373==   total heap usage: 3 allocs, 3 frees, 632 bytes allocated
==373==
==373== All heap blocks were freed -- no leaks are possible
==373==
==373== For lists of detected and suppressed errors, rerun with: -s
==373== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
Cristian@DESKTOP-ST8LL5D:/mnt/c/users/crist/OneDrive/Escritorio/UdeA/50/Labs/Lab03-50-Cristian-Florez$

```

Errores y fugas: La herramienta reporta 0 errores y 0 fugas de memoria (leaks). "All heap blocks were freed" indica que existe una correspondencia exacta entre las solicitudes de reserva de memoria (malloc, realloc, etc.) y las solicitudes de liberación (free). Específicamente, el resumen muestra 3 allocs y 3 frees. En términos de gestión de memoria, significa que el ciclo de vida de los objetos dinámicos fue gestionado correctamente y no quedó memoria huérfana en el heap al finalizar el proceso.

2. ¿Por qué se usa sizeof(int) en lugar del valor literal 4? ¿Qué ventaja ofrece la portabilidad entre arquitecturas?

El uso del operador sizeof(int) es una práctica de programación robusta preferida sobre el valor literal 4 por las siguientes razones:

- Portabilidad: El tamaño de un tipo de dato int no es universal; depende de la arquitectura del procesador (16, 32 o 64 bits) y del modelo de datos del compilador. Mientras que en la mayoría de sistemas actuales es de 4 bytes, en sistemas embebidos o arquitecturas antiguas puede ser de 2 bytes.
- Mantenibilidad: Si en el futuro se decide cambiar el tipo de dato del arreglo (por ejemplo, a long o double), el uso de sizeof actualiza automáticamente el cálculo del espacio necesario durante la compilación, evitando errores de desbordamiento o desperdicio de memoria.

3. ¿Qué devuelve malloc cuando no hay memoria disponible? ¿Por qué es crítico verificar ese valor antes de usarlo?

Cuando el sistema operativo no puede satisfacer una solicitud de memoria (debido a la fragmentación del heap o al agotamiento de la memoria física/swap), la función malloc devuelve un puntero nulo (NULL).

Es crítico verificar si el valor devuelto es NULL antes de intentar desreferenciar el puntero por las siguientes razones:

- Prevención de Fallos (Crash): Intentar escribir o leer en una dirección NULL provoca una violación de acceso o Segmentation Fault, lo que termina abruptamente la ejecución del programa.

- Robustez del Software: En sistemas críticos, el programa debe ser capaz de manejar la falta de memoria de forma controlada (por ejemplo, guardando datos actuales o cerrando conexiones de forma segura) en lugar de colapsar inesperadamente.

2.3 Actividad: Código con bugs de memoria

Compile y ejecute el programa:

```
gcc -Wall -g -o buggy_mem buggy_mem.c
valgrind --leak-check=full --track-origins=yes ./buggy_mem
```

```
cristian@DESKTOP-5T8LL5D:/mnt/c/users/crist/OneDrive/Escritorio/UdeA/SO/Labs/Lab03-SO-Cristian-Florez$ gcc -Wall -g -o buggy_mem buggy_mem.c
buggy_mem.c: In function 'main':
buggy_mem.c:18:5: warning: pointer 'p' used after 'free' [-Wuse-after-free]
   18 |     printf("p[0] = %d\n", p[0]); /* acceso ilegal */
      |     ~~~~~^~~~~~
buggy_mem.c:17:5: note: call to 'free' here
   17 |     free(p);
      |     ~~~~~^
cristian@DESKTOP-5T8LL5D:/mnt/c/users/crist/OneDrive/Escritorio/UdeA/SO/Labs/Lab03-SO-Cristian-Florez$
```

```
cristian@DESKTOP-5T8LL5D:/mnt/c/users/crist/OneDrive/Escritorio/UdeA/SO/Labs/Lab03-SO-Cristian-Florez$ valgrind --leak-check=full --track-origins=yes ./buggy_mem
==381== Memcheck, a memory error detector
==381== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==381== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==381== Command: ./buggy_mem
==381==
==381== error calling PR_SET_PTRACER, vgdb might block
==381== Invalid write of size 4
==381==    at 0x1091E3: main (buggy_mem.c:10)
==381==    Address 0x4a74054 is 0 bytes after a block of size 20 alloc'd
==381==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==381==    by 0x1091BE: main (buggy_mem.c:8)
==381==
hole mundo
==381== Invalid read of size 4
==381==    at 0x109231: main (buggy_mem.c:18)
==381==    Address 0x4a74040 is 0 bytes inside a block of size 20 free'd
==381==    at 0x484988F: free (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==381==    by 0x10922C: main (buggy_mem.c:17)
==381==
Block was alloc'd at
==381==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==381==    by 0x1091BE: main (buggy_mem.c:8)
==381==
p[0] = 0
==381==
==381== HEAP SUMMARY:
==381==    in use at exit: 100 bytes in 1 blocks
==381==    total heap usage: 3 allocs, 2 frees, 632 bytes allocated
==381==
==381== 100 bytes in 1 blocks are definitely lost in loss record 1 of 1
==381==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==381==    by 0x1091F8: main (buggy_mem.c:13)
==381==
==381== LEAK SUMMARY:
==381==    definitely lost: 100 bytes in 1 blocks
==381==    indirectly lost: 0 bytes in 0 blocks
==381==    possibly lost: 0 bytes in 0 blocks
==381==    still reachable: 0 bytes in 0 blocks
==381==    suppressed: 0 bytes in 0 blocks
==381==
==381== For lists of detected and suppressed errors, rerun with: -s
==381== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 0 from 0)
cristian@DESKTOP-5T8LL5D:/mnt/c/users/crist/OneDrive/Escritorio/UdeA/SO/Labs/Lab03-SO-Cristian-Florez$
```

2.4 Actividad: Identificar y corregir errores de memoria

1. Transcriba los mensajes que arroja Valgrind. ¿Cuál mensaje corresponde a cada uno de los tres errores clásicos?

Error 1: Desbordamiento de Búfer (Heap Buffer Overflow)

- Mensaje: Invalid write of size 4 ... Address 0x... is 0 bytes after a block of size 20 alloc'd.

- Contexto: Ocurre en la línea 10 de buggy_mem.c. Indica que el programa intentó escribir datos fuera de los límites del bloque de 20 bytes asignado, accediendo a una posición de memoria inmediatamente posterior a la reserva legítima.

Error 2: Uso de Memoria Después de Liberar (Use-after-free)

- Mensaje: Invalid read of size 4 ... Address 0x... is 0 bytes inside a block of size 20 free'd.
- Contexto: Ocurre en la línea 18. Valgrind detecta que el programa intenta leer el valor p[0] de un bloque que ya fue retornado al sistema mediante free() en la línea 17.

Error 3: Fuga de Memoria (Memory Leak)

- Mensaje: 100 bytes in 1 blocks are definitely lost ... alloc'd at ... main (buggy_mem.c:13).
- Contexto: Indica que la memoria reservada en la línea 13 nunca fue liberada. Valgrind clasifica esto como "definitivamente perdida" porque al finalizar el proceso no existe ningún puntero que apunte a dicha dirección, imposibilitando su liberación manual.

2. Corrija el programa (buggy_mem_fixed.c) y verifique con Valgrind que no queda ningún error ni fuga.

```

cris@DESKTOP-5T8LLSD:/mnt/c/users/crist/OneDrive/Escritorio/Udea/50/Labs/Lab03-50-Cristian-Florez$ gcc -Wall -g -o buggy_mem buggy_mem.c
cris@DESKTOP-5T8LLSD:/mnt/c/users/crist/OneDrive/Escritorio/Udea/50/Labs/Lab03-50-Cristian-Florez$ valgrind --leak-check=full --track-origins=yes ./buggy_mem
==387== Memcheck, a memory error detector
==387== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==387== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==387== Command: ./buggy_mem
==387==
==387== error calling PR_SET_PTRACER, vgdb might block
Programa finalizado sin errores de memoria.
==387==
==387== HEAP SUMMARY:
==387==   in use at exit: 0 bytes in 0 blocks
==387==   total heap usage: 3 allocs, 3 frees, 632 bytes allocated
==387==
==387== All heap blocks were freed -- no leaks are possible
==387==
==387== For lists of detected and suppressed errors, rerun with: -s
==387== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
cris@DESKTOP-5T8LLSD:/mnt/c/users/crist/OneDrive/Escritorio/Udea/50/Labs/Lab03-50-Cristian-Florez$

```

3. ¿Qué consecuencias puede tener un use-after-free en un programa real en términos de seguridad y estabilidad del sistema?

- Estabilidad del Sistema: Puede causar cierres inesperados (crashes) o, de manera más insidiosa, una corrupción de datos silenciosa. Si el gestor de memoria asigna ese bloque liberado a otra variable distinta, el programa original seguirá modificando esa dirección, alterando valores de variables que no tienen relación alguna, lo que produce comportamientos erráticos difíciles de depurar.
- Seguridad Informática: Es una vulnerabilidad explotable. Un atacante puede forzar al sistema a colocar datos específicos (como una dirección de salto o

código malicioso) en el bloque liberado. Cuando el programa original accede al puntero colgante pensando que es un dato legítimo, puede ser desviado para ejecutar código arbitrario o permitir una escalada de privilegios.

3 Traducción de direcciones — Base & Bounds

3.2 Actividad: Base & Bounds — Análisis

1. Compile y ejecute. Muestre la salida completa. ¿Qué ocurre al acceder a VA=64 y VA=100 en el Proceso A? ¿Que haría el SO real ante esta excepción?

```
crstian@DESKTOP-5T8LL5D:/mnt/c/users/crist/OneDrive/Escritorio/UdeA/SO/Labs/Lab03-SO-Cristian-Florez$ ./base_bounds
--- Proceso A (base=32, bounds=64) ---
VA= 0 -> PA= 32
VA= 10 -> PA= 42
VA= 63 -> PA= 95
[EXCEPCION] VA=64 viola bounds=64
[EXCEPCION] VA=100 viola bounds=64
--- Proceso B (base=128, bounds=80) ---
VA= 0 -> PA=128
VA= 10 -> PA=138
VA= 63 -> PA=191
VA= 64 -> PA=192
[EXCEPCION] VA=100 viola bounds=80
crstian@DESKTOP-5T8LL5D:/mnt/c/users/crist/OneDrive/Escritorio/UdeA/SO/Labs/Lab03-SO-Cristian-Florez$
```

Para el Proceso A (Base=32, Bounds=64), el programa arroja una [EXCEPCIÓN] tanto para VA=64 como para VA=100.

Ante esta excepción de hardware (fuera de límites), un Sistema Operativo real enviaría una señal de terminación al proceso

2. Agregue un Proceso C (base=0, bounds=32) al programa y traduzca las mismas VAs. ¿Puede el Proceso A acceder a las direcciones del Proceso C directamente? Justifique.

No puede acceder ya que el aislamiento es absoluto debido a la traslación de direcciones. El Proceso A solo puede generar direcciones físicas a partir de su base (32) hacia arriba. No existe ninguna dirección virtual (VA) que el Proceso A pueda generar que, sumada a su base, resulte en una dirección física menor a 32. El hardware asegura que cada proceso esté encerrado en su propio rango físico definido por sus registros específicos.

3. ¿Cuál es la limitación principal del esquema base & bounds que motiva el surgimiento de la segmentación?

La limitación principal es la Fragmentación Interna y la Inflexibilidad.

- Fragmentación Interna: Este esquema obliga a que el espacio de direcciones sea un bloque contiguo en la memoria física. Si un proceso tiene un espacio de direcciones grande pero solo usa una pequeña parte, ese espacio vacío debe existir físicamente en la RAM, desperdiciando memoria.
- Motivación de la Segmentación: La segmentación surge para permitir que diferentes partes del espacio de direcciones (Código, Heap, Stack) se ubiquen en lugares distintos de la memoria física, permitiendo que el espacio vacío entre el heap y el stack no ocupe RAM física real.

4 Segmentación

1. Muestre el cálculo paso a paso para cada VA.

A. VA = 0x03A0

BINARIO = 00 0011 1010 0000

SELECTOR = 00 (CODE), BASE 0x4000

TAMAÑO = 2KB (2048 B)

OFFSET = 0x3A0 (928 DECIMAL)

VALIDACIÓN = 928 < 2048 (ES VÁLIDO)

PA = BASE + OFFSET = 0x4000 + 0x3A0
= 0x43A0

B. VA = 0x1B00

BINARIO = 01 1000 0000 0000

SELECTOR = 01 (HEAP) BASE 0x6000

TAMAÑO = 8KB (8192 B)

OFFSET = 0x800 (2048 DECIMAL)

VALIDACIÓN = 2048 < 8192 (VÁLIDO)

PA = BASE + OFFSET = 0x6000 + 0x800
= 0x6800

C. VA = 0x3C00 (STACK - CRECIMIENTO NEGATIVO)

BINARIO = 11 11 00 0000 0000

SELECTOR = 11 (STACK) BASE = 0x2800

TAMAÑO = 2KB (2048B)

OFFSET = 0xC00 (3072 DECIMAL)

AJUSTE NEGATIVO PORQUE EL STACK
CRECE HACIA ABAJO =

$$3072 - 4096 = -1024$$

VALIDACIÓN = $|-1024| \leq 2048$ (VALIDO)

PA = BASE + OFFSET NEGATIVO

$$= 10240 + (-1024) = 9216 = 0x2400$$

D. VA = 0x0C00

BINARIO = 00 11 00 0000 0000

SELECTOR = 00 (CODE) BASE = 0x4000

TAMAÑO = 2KB (2048B)

OFFSET = 0xC00 (3072 DECIMAL)

VALIDACIÓN = $3072 > 2048$ (NO VALIDO)

FUERA DE LOS LIMITES

E. VA = 0x2200

BINARIO = 10 00 10 0000 0000

SELECTOR = 10 (NO DEFINIDO)

RESULTADO = SELEMENTO NO VALIDO

2. ¿Por qué el Stack crece en dirección negativa? ¿Qué ajuste especial requiere la fórmula al calcular el PA?

Por convención y eficiencia. Al colocar el Heap en la parte baja y el Stack en la parte alta del espacio virtual creciendo uno hacia el otro, se permite que el espacio libre entre ambos se use dinámicamente según la necesidad de la aplicación.

Ajuste especial: En segmentos negativos, el hardware calcula el offset negativo restando el tamaño máximo direccionable ($2^{\text{bits de offset}}$) al offset extraído. La

dirección física se obtiene sumando este valor negativo (que actúa como una resta) a la base física, la cual apunta al final del segmento en lugar del inicio.

3. ¿Qué ventaja tiene la segmentación frente a base & bounds en cuanto a utilización de la memoria física?

La principal ventaja es la utilización eficiente de la memoria física y el soporte de espacios dispersos.

- En Base & Bounds, el hueco vacío entre el heap y el stack debe ocupar espacio en la RAM física de forma continua.
- En Segmentación, solo se mapean a la RAM los segmentos que realmente contienen datos (Code, Heap, Stack). El espacio en blanco virtual no consume memoria física real, permitiendo ejecutar más procesos simultáneamente.

4. ¿Qué es la fragmentación externa? ¿Por qué surge con segmentación?

Es el fenómeno donde la memoria física disponible está dividida en muchos huecos pequeños dispersos entre los segmentos de diferentes procesos.

Surge debido a que los segmentos tienen tamaños variables y se asignan/liberan en momentos distintos. Con el tiempo, aunque la suma total de los huecos sea suficiente para un nuevo segmento, es posible que no exista un bloque contiguo lo suficientemente grande para alojarlo.

5 Paginación

1. ¿Cuántos bits se necesitan para el VPN y Cuántos para el offset? Muestre el cálculo.

Offset: El tamaño de la página es de 4 KB, lo cual equivale a 2^{12} bytes. Por lo tanto, se requieren 12 bits para el offset (estos bits permiten direccionar cada byte dentro de una página).

VPN: El espacio virtual es de 32 bits. Si restamos los bits del offset, tenemos:

$$\text{VPN} = \text{Total bits VA} - \text{bits offset} = 32 - 12 = 20 \text{ bits}$$

Cálculo: Con 20 bits para el VPN, el sistema puede gestionar hasta $2^{20} = 1,048,576$ páginas virtuales por proceso.

2. ¿Cuántas entradas tiene la tabla de páginas de un proceso?

Dado que cada página virtual necesita una entrada en la tabla (PTE) para indicar su ubicación física, la cantidad de entradas es igual al número total de páginas virtuales:

$$\text{Entradas} = 2^{\text{bits VPN}} = 2^{20} = 1,048,576 \text{ entradas}$$

3. ¿Cuánta memoria ocupa la tabla de páginas completa? ¿Es razonable ese tamaño para cada proceso?

Se multiplica el número de entradas por el tamaño de cada entrada (PTE = 4 bytes):

$$\text{Tamaño} = 1,048,576 \times 4 \text{ bytes} = 4,194,304 \text{ bytes} = 4 \text{ MB}$$

No es razonable para todos los casos. Aunque 4 MB parece poco para una PC moderna, este espacio se requiere por cada proceso. Si se ejecutan 100 procesos, se gastarían 400 MB solo en tablas de páginas, lo cual es un desperdicio significativo de RAM física, especialmente en sistemas con recursos limitados.

4. ¿Cuántos bits necesita el PFN dentro de la PTE? ¿Qué información almacenan los bits restantes? Mencione al menos 3 bits de control y su función.

Bits del PFN: El espacio físico es de 20 bits. Como el tamaño de marco (frame) es igual al de la página (4 KB = 12 bits de offset), el número de bits para el PFN es:

$$\text{PFN} = \text{Total bits PA} - \text{bits offset} = 20 - 12 = 8 \text{ bits}$$

Información restante: De los 32 bits de la PTE (4 bytes), solo 8 se usan para el PFN. Los 24 bits restantes se utilizan para metadatos y bits de control.

Bits de control:

1. Valid Bit (V): Indica si la página está actualmente en el espacio de direcciones virtual del proceso (si es legal acceder a ella).
2. Protection Bits (R/W/X): Establecen los permisos de la página (lectura, escritura o ejecución).
3. Dirty Bit (D): Indica si la página ha sido modificada desde que se cargó en RAM; es vital para decidir si debe escribirse en disco antes de ser reemplazada.

5.2 Actividad: Simulador de paginación

1. Compile y ejecute el simulador. Muestre la salida completa.

```
cristian@DESKTOP-5T8LLSD:/mnt/c/users/crist/OneDrive/Escritorio/UdeA/SO/Labs/Lab03-SO-Cristian-Florez$ ./paging_sim
VA      VPN      Offset    PFN      PA
-----
VA=0x00 VPN= 0 Offset= 0 -> PFN= 3 PA=0x30
VA=0x0F VPN= 0 Offset=15 -> PFN= 3 PA=0x3F
VA=0x20 VPN= 2 Offset= 0 -> PFN= 7 PA=0x70
VA=0x35 VPN= 3 Offset= 5 -> PFN= 2 PA=0x25
VA=0x10 VPN= 1 Offset= 0 -> PAGE FAULT (pagina no presente)
VA=0xA3 VPN=10 Offset= 3 -> PFN= 4 PA=0x43
VA=0xC8 VPN=12 Offset= 8 -> PFN= 6 PA=0x68
VA=0xF0 VPN=15 Offset= 0 -> PAGE FAULT (pagina no presente)
```

2. ¿Qué ocurre con las VAs 0x10 y 0xA3? ¿Qué debería hacer el SO real ante un page fault?

VA 0x10 (VPN 1): El simulador reporta un PAGE FAULT. Esto ocurre porque, aunque la dirección es parte del espacio virtual del proceso, el bit de presencia (present bit) en la tabla de páginas está en 0.

VA 0xA3 (VPN 10): Se traduce correctamente a la dirección física 0x43 (PFN 4 + Offset 3).

Ante un fallo de página, el SO debe:

1. Interrumpir el proceso y cambiar a modo kernel (trap).
2. Consultar la estructura de datos del proceso para ver si la página reside en el disco (área de intercambio o swap).
3. Si la página es válida pero no está en RAM, debe buscar un marco físico libre, cargar la página desde el disco, actualizar la tabla de páginas (PFN y bit de presencia) y reanudar la instrucción que causó el fallo.

3. ¿Cuántos accesos a memoria física requiere completar una instrucción load con tabla de páginas de un solo nivel? ¿Por qué es costoso y qué solución de hardware existe?

Con una tabla de páginas de un solo nivel, una instrucción load requiere dos accesos a la memoria física:

1. Un acceso para leer la tabla de páginas (PTE) y obtener la traducción (PFN).
2. Un segundo acceso para leer el dato real en la dirección física calculada.

Este proceso es costoso porque la memoria RAM es órdenes de magnitud más lenta que la CPU, duplicando el tiempo de ejecución de cada instrucción. La solución de hardware es el TLB (Translation Lookahead Buffer), una caché de alta velocidad que almacena las traducciones recientes, permitiendo que la mayoría de las veces el primer acceso (a la tabla) sea instantáneo.

4. ¿Qué ventaja tiene la paginación sobre la segmentación en cuanto al fenómeno de fragmentación?

En la segmentación, los segmentos tienen tamaños variables, lo que deja "huecos" libres en la RAM que a veces son demasiado pequeños para nuevos segmentos. Al dividir la memoria en trozos de tamaño fijo (marcos), cualquier página virtual puede caber en cualquier marco físico disponible. No importa si los marcos están dispersos en la RAM; el SO siempre puede satisfacer una solicitud de memoria siempre que haya suficientes marcos libres, maximizando el aprovechamiento de la memoria física.

6 Gestión de espacio libre

6.1 Actividad: Simulación de estrategias de asignación

1. Para cada solicitud indique que bloque asigna first fit. Muestre la lista libre resultante tras las 4 asignaciones.

Estrategia: Recorre la lista y asigna el primer bloque que sea lo suficientemente grande.

1. malloc(212): El primero que sirve es el Bloque B (500). Queda un fragmento de $500 - 212 = 288$ B.
2. malloc(417): El primero que sirve es el Bloque E (600). Queda un fragmento de $600 - 417 = 183$ B.
3. malloc(98): El primero que sirve es el Bloque A (100). Queda un fragmento de $100 - 98 = 2$ B.
4. malloc(426): Falla. No hay ningún bloque restante mayor o igual a 426 (quedan bloques de 2, 288, 200, 300 y 183).

Lista Libre Resultante (First Fit):

- 0x0162: 2 B
- 0x02D4: 288 B
- 0x0400: 200 B
- 0x0500: 300 B
- 0x08A1: 183 B

2. Repita con best fit. ¿Cambia el resultado?

Busca en toda la lista el bloque más pequeño que sea mayor o igual al tamaño solicitado.

1. malloc(212): El mejor es el Bloque D (300). Queda un fragmento de $300 - 212 = 88$ B.
2. malloc(417): El mejor es el Bloque B (500). Queda un fragmento de $500 - 417 = 83$ B.
3. malloc(98): El mejor es el Bloque A (100). Queda un fragmento de $100 - 98 = 2$ B.
4. malloc(426): El mejor es el Bloque E (600). Queda un fragmento de $600 - 426 = 174$ B.

Sí cambia el resultado. Best Fit logró satisfacer las 4 solicitudes, mientras que First Fit falló en la última.

3. ¿Cuál estrategia genera más fragmentación externa en este caso? ¿Cuál la minimiza?

Estrategia que genera más fragmentación: En este caso específico, First Fit generó más fragmentación externa al dejar bloques medianos inservibles que impidieron la última asignación grande.

Estrategia que la minimiza: Best Fit minimizó la fragmentación externa al ajustar mejor las solicitudes a los bloques, preservando el bloque más grande (600 B) para la solicitud más exigente.

4. ¿Qué es el coalescing? Ilustre un caso donde su ausencia provoca que una solicitud de 250 bytes falle aunque haya suficiente memoria total libre.

El coalescing es la técnica de unir dos o más bloques libres contiguos en la memoria física para formar un único bloque más grande.

Caso de falla sin coalescing: Supongamos que tenemos dos bloques libres adyacentes de 150 bytes cada uno (Total libre: 300 B). Si el programa solicita malloc(250), la solicitud fallará si el sistema no realiza coalescing, ya que verá dos unidades pequeñas separadas en lugar de una unidad de 300 bytes capaz de albergar la solicitud.

5. ¿Qué es la fragmentación interna? ¿Cuándo aparece típicamente al usar un slab allocator?

La fragmentación interna ocurre cuando se asigna a un proceso un bloque de memoria ligeramente más grande de lo que solicitó. El espacio extra dentro del bloque queda desperdiciado porque no puede ser usado por nadie más.

Slab Allocator: Se usa típicamente en kernels para objetos de tamaño fijo (ej. descriptores de procesos). La fragmentación interna aparece cuando el tamaño del objeto no es múltiplo exacto del tamaño del "slab" o cuando la estructura del objeto requiere alineación (padding), dejando pequeños espacios muertos dentro de cada celda asignada.

6.3 Actividad: Fragmentación en glibc — Análisis

```
crstian@DESKTOP-5T8LL5D:/mnt/c/users/crist/OneDrive/Escritorio/UdeA/SO/Labs/Lab03-SO-Cristian-Florez$ ./fragmentation
malloc( 16) -> 0x7ffffd43002a0
malloc( 32) -> 0x7ffffd43004d0
malloc( 64) -> 0x7ffffd4300500
malloc(128) -> 0x7ffffd4300550
malloc(256) -> 0x7ffffd43005e0
malloc(512) -> 0x7ffffd43006f0
malloc(1024) -> 0x7ffffd4300900
malloc( 512) -> 0x7ffffd4300d10
malloc( 256) -> 0x7ffffd4300f20
malloc( 128) -> 0x7ffffd4301030

Liberando bloques en índices pares...

malloc(1500) -> 0x7ffffd43010c0 [exito]
crstian@DESKTOP-5T8LL5D:/mnt/c/users/crist/OneDrive/Escritorio/UdeA/SO/Labs/Lab03-SO-Cristian-Florez$
```

1. ¿Son consecutivas en memoria las direcciones asignadas? ¿Qué patrón de separación observa entre bloques contiguos?

No son consecutivas en el sentido estricto de estar pegadas byte a byte. Aunque las direcciones son crecientes, existe un espacio adicional entre el final de un bloque y el inicio del siguiente.

Patrón de separación: en el bloque de 64 bytes (0x...500) y el de 128 bytes (0x...550), la diferencia es de 0x50 (80 bytes decimales). Esto indica que para un bloque de 64 bytes solicitado, glibc reservó 80 bytes. Esos 16 bytes extra corresponden a los metadatos del bloque (tamaño, flags de estado) y al alineamiento de memoria (usualmente a 8 o 16 bytes). glibc necesita estos metadatos para saber cuánto liberar cuando se llama a free().

2. ¿Tiene éxito la asignación final de 1500 bytes? Explique el resultado en términos de fragmentación.

La asignación de 1500 bytes tuvo éxito (0x7ffffd43010c0). A pesar de que se liberaron bloques en índices pares, la suma de esos huecos dispersos no necesariamente formó un espacio contiguo de 1500 bytes (fragmentación externa). Sin embargo, el asignador malloc tiene dos opciones:

1. Intentar el coalescing (fusión) de bloques libres contiguos para crear un espacio más grande.
2. Si la memoria libre fragmentada no es suficiente o no se puede consolidar, el asignador solicita más memoria al sistema operativo mediante la llamada al

sistema sbrk o mmap, expandiendo el heap hacia arriba para ubicar el nuevo bloque de 1500 bytes.

3. Consulta: ¿Cuál es la diferencia entre el allocator de usuario (malloc/glibc) y el del kernel (buddy system, slab)? ¿Por qué existen dos niveles de gestión de memoria?

Característica	Allocator de Usuario (glibc/malloc)	Allocator del Kernel (Buddy / Slab)
Objetivo	Gestionar memoria para variables y estructuras del proceso.	Gestionar memoria para estructuras del SO (nodos, buffers, procesos).
Estrategia	Optimiza para evitar fragmentación y velocidad en espacio de usuario.	Buddy System: Evita fragmentación externa. Slab: Evita fragmentación interna en objetos comunes.
Granularidad	Maneja bytes arbitrarios (ej. 17 bytes, 1024 bytes).	Maneja páginas completas o tamaños fijos predefinidos.

7.1 Actividad: Localidad y TLB — Análisis

```
crstian@DESKTOP-5T8LL5D:/mnt/c/users/crist/OneDrive/Escritorio/UdeA/SO/Labs/Lab03-SO-Cristian-Florez$ ./tlb_locality
Secuencial : 16.12 ms (sum=8796090925056)
Aleatorio : 102.78 ms (sum=8796090925056)
```

1. ¿Cuántas veces más lento es el acceso aleatorio frente al secuencial? Muestre el promedio de 3 ejecuciones de tlb locality.

Ejecución	Secuencial (ms)	Aleatorio (ms)	Factor de diferencia
1	16.12	102.78	6.37x
2	15.95	105.12	6.59x
3	16.05	101.45	6.32x
Promedio	16.04 ms	103.11 ms	6.42x

2. Explique con el modelo del TLB por qué el acceso aleatorio es más lento. ¿Qué ocurre con el hit rate en cada caso?

La diferencia de rendimiento se explica por el principio de localidad espacial:

- Acceso Secuencial: Cuando el programa accede a la memoria de forma lineal, una sola entrada cargada en el TLB permite traducir las direcciones de miles de enteros consecutivos (en una página de 4 KB caben 1024 enteros de 4 bytes).
 - Hit Rate: Es extremadamente alto (99.9). Casi todas las traducciones se encuentran en el TLB (caché), minimizando los accesos a la tabla de páginas en RAM.
- Acceso Aleatorio: Al saltar a direcciones arbitrarias, es muy probable que cada acceso apunte a una página virtual distinta. Si el número de páginas accedidas supera la capacidad del TLB (ej. 64 entradas), se producen constantes fallos.
 - Hit Rate: Es muy bajo. El hardware debe realizar un Page Table Walk (consultar la RAM física) para casi cada instrucción load, lo que aumenta drásticamente el tiempo de ejecución.

3. Si el tamaño de página fuera 64 KB en lugar de 4 KB, ¿mejoraría o empeoraría la situación con accesos aleatorios? Justifique desde el punto de vista del TLB y del uso de memoria.

Si se incrementara el tamaño de página de 4 KB a 64 KB, se observarían los siguientes efectos:

- Desde el punto de vista del TLB (Mejora): La situación mejoraría significativamente. Con el mismo número de entradas (64), el TLB podría cubrir 4 MB de memoria (64 X 64 KB) en lugar de solo 256 KB. Esto aumentaría la probabilidad de que una dirección aleatoria caiga dentro de una página ya mapeada en el TLB, incrementando el hit rate.
- Desde el punto de vista del uso de memoria (Empeora): Aparecería una mayor fragmentación interna. Si un proceso solo necesita almacenar una pequeña cantidad de datos (ej. 1 KB), el sistema operativo igual debe asignarle una página completa de 64 KB, desperdiciando 63 KB de memoria física que no podrá ser utilizada por otros procesos.

7.2 Actividad: Comportamiento de los TLB

Responda conceptualmente estas preguntas:

1. Un TLB con 64 entradas (fully associative) y páginas de 4 KB. ¿Cuánta memoria puede cubrir sin generar misses? ¿Es suficiente para un proceso moderno típico?

La memoria total que puede cubrir el TLB antes de generar un miss se calcula multiplicando el número de entradas por el tamaño de la página:

$$64 \text{ entradas} \times 4 \text{ KB/página} = 256 \text{ KB}$$

Esta capacidad es claramente insuficiente para un proceso moderno típico. Las aplicaciones actuales (navegadores, bases de datos o entornos de ejecución como la JVM) manejan gigabytes de datos y tienen conjuntos de trabajo (working sets) mucho mayores a 256 KB. Con esta cobertura, el sistema experimentaría constantes TLB misses, lo que obligaría al hardware o al sistema operativo a realizar caminatas por las tablas de páginas (page table walks) en la RAM, degradando el rendimiento general.

2. Consulte: ¿Qué es un TLB shutdown y en qué situación ocurre en sistemas multiprocesador? ¿Por qué es una operación costosa?

Un TLB Shutdown es una operación de sincronización que ocurre cuando un núcleo de la CPU invalida una entrada en las tablas de páginas y debe forzar a todos los demás núcleos a que también eliminen esa traducción de sus TLB locales.

Ocurre cuando se modifica la estructura de la memoria compartida, por ejemplo, al liberar memoria con `free()`, al realizar swapping (mover una página al disco) o al cambiar los permisos de una región de memoria.

Es extremadamente costosa porque requiere Interrupciones entre Procesadores (IPI). El núcleo que inicia el cambio debe pausar la ejecución de los otros núcleos, esperar a que estos limpien su TLB y recibir una confirmación (ACK). Esto introduce una latencia significativa y rompe el paralelismo del sistema.

3. Explique la diferencia entre TLB gestionado por hardware (CISC/x86) y por software (RISC/MIPS). ¿Cuál ofrece mayor flexibilidad al diseñador del SO y por qué?

Gestión por Hardware (x86): El hardware es responsable de manejar los TLB misses. Cuando una traducción no está en el TLB, la MMU (unidad de gestión de memoria) recorre automáticamente las tablas de páginas en RAM. El SO solo debe configurar la ubicación de estas tablas en registros específicos (como CR3).

Gestión por Software (MIPS/RISC): Ante un TLB miss, el hardware genera una excepción y le entrega el control al sistema operativo. El kernel debe buscar la traducción manualmente en sus estructuras de datos e insertarla en el TLB usando instrucciones especiales.

Flexibilidad para el Diseñador del SO: La gestión por software ofrece una flexibilidad mucho mayor. Debido a que el hardware no impone un formato rígido para las tablas de páginas, el diseñador del SO puede implementar cualquier estructura de datos (tablas invertidas, árboles, tablas de hash) para gestionar la memoria virtual, optimizándola según las necesidades específicas de la carga de trabajo sin estar limitado por los requisitos de la CPU.